Question 1

The general function-approximation TD update is:

$$\theta_k \leftarrow \theta_k + \varepsilon\left[r_t + \gamma\hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)\right]\frac{\partial f(s_t, \boldsymbol{\theta})}{\partial \theta_k} \tag{1}$$

To recover tabular TD, let $\boldsymbol{\theta}$ have one entry per state and define $f(s, \boldsymbol{\theta}) = \sum_j \theta_j \mathbb{1}(s = j)$. Then $\frac{\partial f}{\partial \theta_k} = \mathbb{1}(s_t = k)$, giving:

$$\theta_k \leftarrow \theta_k + \varepsilon\left[r_t + \gamma\hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)\right]\mathbb{1}(s_t = k) \tag{2}$$

Only the parameter for the current state is updated, which is exactly lookup-table TD(0).

Question 2

For the linear approximator $f(s, \boldsymbol{w}) = \sum_k w_k \varphi_k(s)$, the partial derivative is simply $\frac{\partial f}{\partial w_k} = \varphi_k(s_t)$. Substituting into Eq. (1):

$$w_k \leftarrow w_k + \varepsilon\underbrace{\left[r_t + \gamma\hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)\right]}_{\delta_t}\varphi_k(s_t) \tag{3}$$

Each weight update is the product of two factors: a scalar TD error δ_t (shared across all k) and the feature activation $\varphi_k(s_t)$. Features that are active in the current state absorb the prediction error; inactive features are unchanged. This enables generalisation: states sharing features transfer learned value.

Question 3

We implement TD with a tapped delay line representation ($\varphi_{i\tau}(\boldsymbol{S}) = S_{i\tau}$) and the linear update from Q2. Figure 1 shows the task inputs: a spike stimulus at $t = 10$ s and a Gaussian reward centred at $t = 20$ s.

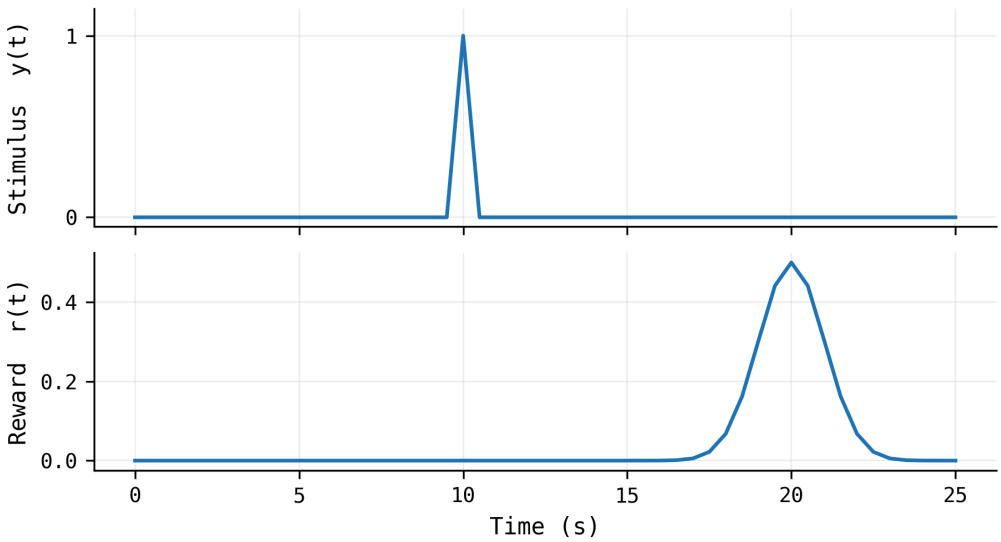

Figure 1: Stimulus and reward as functions of time.

Figure 2 shows three key observables recorded every 10th trial (colour runs from dark = trial 1 to light = trial 201):

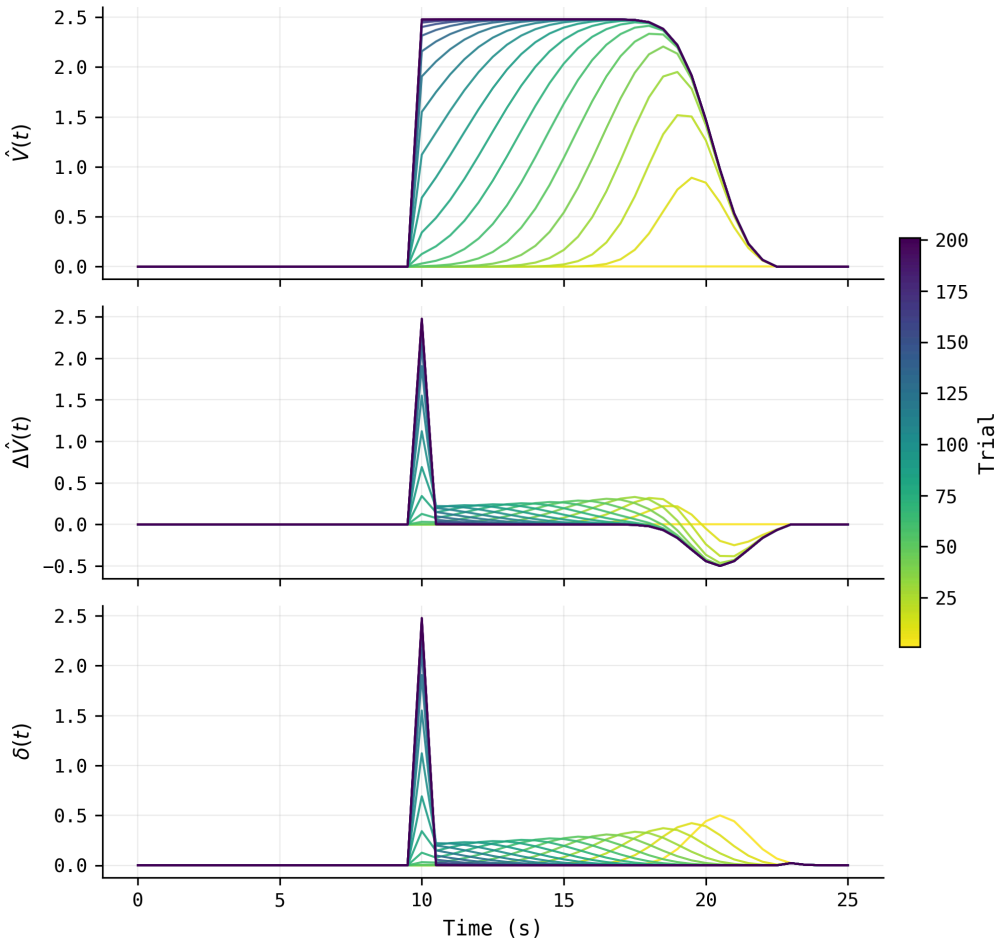

Figure 2: $\hat{V}(t)$, $\Delta\hat{V}(t)$, and $\delta(t)$ across 21 sampled trials.

Observations.

- *Early trials*: δ peaks at reward time because the model has not yet assigned value to earlier states.
- *Mid-learning*: TD bootstrapping propagates value backward in time, building an anticipatory ramp in $\hat{V}$ after the stimulus.
- *Late trials*: δ shifts to stimulus onset while the reward-time error vanishes as the reward becomes fully predicted.

Dopamine correspondence. In the TD theory, dopamine neuron firing corresponds to $\delta(t)$, the prediction error, not the value $\hat{V}(t)$. The simulation reproduces the experimentally observed cue-ward shift: the δ bump migrates from reward time to stimulus time over the course of learning, with no intermediate activity between the two events.

Appendix: AI Disclosure

This report was drafted by hand and refined with AI assistance as detailed below. All simulation were verified against the handout; all results were inspected and interpreted by the author.

GitHub Copilot — autocomplete during report writing, primarily for equation typesetting.

Claude Code — the following prompts were used:

- "read src/coursework/assignment.pdf and give me a tip for question 1"
- "give me a hint for question 2"
- "walk me through example of value functions from reinforcement learning"
  - "how do we go from this to TD learning?"
  - "explain question 3"
- "refine the report so far"
- " src/lib/core.py defines an experiment, add the plots needed by src/pdfs/assignment.pdf question 3 as publication quality matplotlib plots with monospace font, and save the plots to src/report/figures/question-3"
- "read the assignment and report and bugs in the code"
- "in src/lib/core.py properly parameterise run_td_tapped to support both tapped and boxcar representations."
- "add the plots for question-4 in the usual way to src/lib/core.py"
- "now add in question-5, same as question-4 except reward_probability=0.5, n_trials=1000 and seed=11"
- "now for question-6 do a sweep of the reward probabilities of [0.0,0.25,0.5,0.75,1.0] and make the appropriate plots"